

Surprise Quiz-4 : Solutions

1. Consider the following directed graph $G = (V, E)$:

(node : [neighbor₁(w₁), neighbor₂(w₂), ...])

$s : [a(4), b(5)]; a : [b(-2), c(4)]; b : [c(3)]; c : []$

Run the Bellman-Ford algorithm from source s .

Let $d(v)$ denote the shortest distance estimate to vertex v .

What is the value of $d(c)$ after **2 iterations** of the algorithm?

Solution: 5

Explanation:

Bellman-Ford Algorithm Solution

1. Graph Representation

Edges:

- $s \rightarrow a$ with weight 4
- $s \rightarrow b$ with weight 5
- $a \rightarrow b$ with weight -2
- $a \rightarrow c$ with weight 4
- $b \rightarrow c$ with weight 3

$$E = \{(s, a), (s, b), (a, b), (a, c), (b, c)\}$$

2. Initialization

$$d(s) = 0, \quad d(a) = \infty, \quad d(b) = \infty, \quad d(c) = \infty$$

3. Relaxation Rule

$$d(v) = \min(d(v), d(u) + w(u, v))$$

4. Iteration 1

$$d(a) = \min(\infty, 0 + 4) = 4$$

$$d(b) = \min(\infty, 0 + 5) = 5$$

$$d(b) = \min(5, 4 - 2) = 2$$

$$d(c) = \min(\infty, 4 + 4) = 8$$

$$d(c) = \min(8, 2 + 3) = 5$$

After Iteration 1:

$$d(s) = 0, \quad d(a) = 4, \quad d(b) = 2, \quad d(c) = 5$$

5. Iteration 2

$$d(a) = \min(4, 0 + 4) = 4$$

$$d(b) = \min(2, 0 + 5) = 2$$

$$d(b) = \min(2, 4 - 2) = 2$$

$$d(c) = \min(5, 4 + 4) = 5$$

$$d(c) = \min(5, 2 + 3) = 5$$

No further updates occur.

6. Table of Distances

Iteration	$d(s)$	$d(a)$	$d(b)$	$d(c)$
0	0	∞	∞	∞
1	0	4	2	5
2	0	4	2	5

Final Answer

$$\boxed{d(c) = 5}$$

Why 8 is *not* the correct answer

Many students incorrectly obtain $d(c) = 8$. This happens due to a misunderstanding of how relaxation works within an iteration of the Bellman-Ford algorithm.

The Incorrect Reasoning

Students often compute values in Iteration 1 as follows:

- $d(a) = 4$
- $d(b) = 5$ (from $s \rightarrow b$)

Then they proceed:

- $a \rightarrow c: 4 + 4 = 8$
- $b \rightarrow c: 5 + 3 = 8$

Thus, they conclude:

$$d(c) = 8$$

Why This is Wrong

This reasoning assumes that updates made during the iteration are **not immediately available**. In other words, it treats the algorithm as if all updates happen *simultaneously* at the end of the iteration.

However, this is **not how Bellman-Ford works**.

Correct Concept

In Bellman-Ford, edges are relaxed **sequentially**, and updates are applied **immediately**. This means:

- After processing $a \rightarrow b$, we update:

$$d(b) = 2$$

- This updated value is then used when relaxing $b \rightarrow c$ in the *same iteration*

So instead of using $d(b) = 5$, we use:

$$d(b) = 2$$

Thus:

$$d(c) = 2 + 3 = 5$$

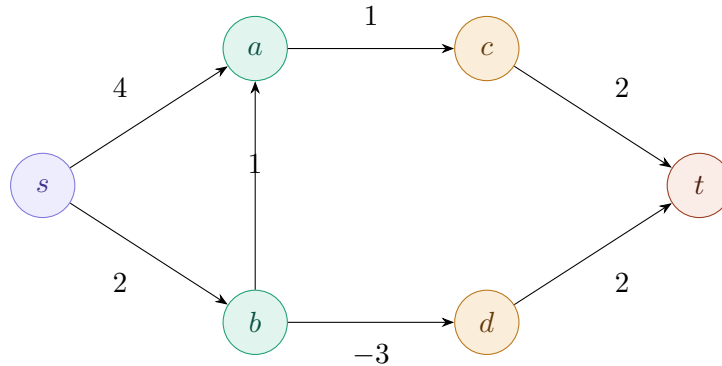
Key Insight

Bellman-Ford allows improvements to propagate within the same iteration. It is **not a level-by-level (layered) algorithm**.

Therefore, 8 is incorrect because it ignores the immediate propagation of improved distances within an iteration.

Question 2, 3

Question 2,3 use the same graph below (source = s , $n = 5$ vertices).



Edges: $s \rightarrow a$ (4), $s \rightarrow b$ (2), $a \rightarrow c$ (1), $b \rightarrow a$ (1), $b \rightarrow d$ (-3), $c \rightarrow t$ (2), $d \rightarrow t$ (2).

- Run Bellman-Ford from source s on G as given above. The algorithm fills an array $A[i, v]$ via:

$$A[0, s] = 0, \quad A[0, v] = +\infty \quad \forall v \neq s,$$

$$A[i, v] = \min\left(A[i-1, v], \min_{(w,v) \in E} \{A[i-1, w] + \ell_{wv}\}\right), \quad i = 1, \dots, n-1.$$

The table below shows $A[i, v]$ for $i = 0, 1, 2$. **Fill in the single missing entry** $A[3, d]$, and state $\text{dist}(s, t)$.

	s	a	b	c	d
$i = 0$	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$
$i = 1$	0	4	2	$+\infty$	$+\infty$
$i = 2$	0	3	2	5	-1
$i = 3$	0	3	2	4	

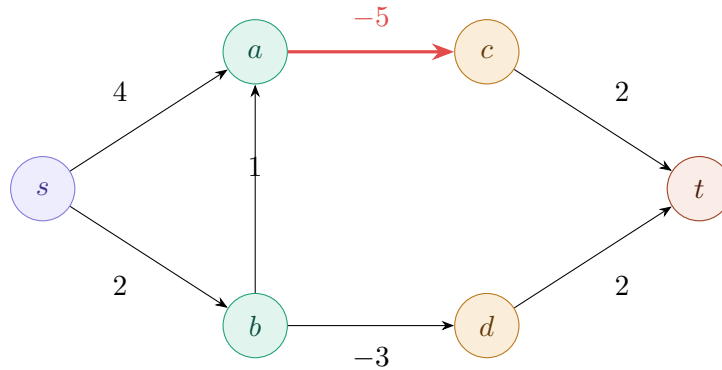
Solution:

$$A[3, d] = \underline{-1}$$

$$\text{dist}(s, t) = \underline{1}$$

Explanation $A[3, d] = 2 - 3 = -1$. $\text{dist}(s, t) = \min(A[3, c] + 2, A[3, d] + 2) = \min(6, -1 + 2) = 1$.

3. **Now change one edge:** replace $a \rightarrow c$ (1) with $a \rightarrow c$ (-5). All other edges remain unchanged.



Part (a). Consider the path $s \rightarrow b \rightarrow a \rightarrow c \rightarrow t$. Compute its total length in the updated graph.

$$\ell(s \rightarrow b \rightarrow a \rightarrow c \rightarrow t) = \underline{\hspace{2cm}}$$

Solution:

$$\ell(s \rightarrow b \rightarrow a \rightarrow c \rightarrow t) = \underline{0}$$

Ans: $2 + 1 + (-5) + 2 = 0$.

Part (b).

Determine whether the updated graph G' has a negative cycle reachable from s . Justify in 1–2 sentences.

Ans: No negative cycle. All paths are simple. There is no cycle among the edges at all (the graph is a DAG once you note no back-edges). The extended BF test will find $A[4, v] = A[5, v]$ for all v , confirming no negative cycle.

4. Consider a weighted directed graph (which may have negative edge weights) and the distance matrix $dist$ computed using the Floyd–Warshall algorithm. How can one detect the presence of a negative weight cycle in the graph?
- (a) If there exists a vertex i such that $dist[i][i] < 0$ after the algorithm terminates.
 - (b) If there exists a pair of vertices (i, j) such that $dist[i][j] = \infty$ after the algorithm terminates.
 - (c) If there exists a pair of vertices (i, j) with $i \neq j$ such that $dist[i][j] < 0$.
 - (d) If the graph contains at least one edge with negative weight.

Solution: A

Explanation: In the Floyd–Warshall algorithm, the entry $dist[i][i]$ represents the shortest distance from a vertex i back to itself. If there are no negative weight cycles, the shortest distance from any vertex to itself is 0. However, if there exists a **negative weight cycle** reachable from vertex i , then the algorithm will keep reducing the value of $dist[i][i]$ as it finds cheaper paths looping through the cycle. As a result, after the algorithm terminates, if $dist[i][i] < 0$ for any vertex i , it indicates the presence of a negative weight cycle.

Other options are incorrect:

- (b) $dist[i][j] = \infty$ simply means that j is not reachable from i , not that there is a negative cycle.
- (c) $dist[i][j] < 0$ does not necessarily imply a negative cycle. It could just be a path with negative total weight.
- (d) The presence of a negative edge does not guarantee a negative cycle.

5. In the Floyd–Warshall algorithm, the distance matrix is updated as:

$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$$

for all vertices i, j, k . Assume the graph contains no negative weight cycles. Which of the following statements are **TRUE**? _____

- (a) After the algorithm terminates, $D[i][j]$ contains the shortest distance from i to j .
- (b) During execution, the entry $D[i][j]$ can increase for some i, j .
- (c) The algorithm works correctly even if there are negative weight edges.
- (d) The recurrence is based on a greedy choice of the next vertex.

Solution: A,C

Explanation:

- **(a) True:** After the Floyd–Warshall algorithm terminates, $D[i][j]$ stores the shortest distance from vertex i to vertex j , provided there are no negative weight cycles in the graph.
- **(b) False:** The value of $D[i][j]$ never increases during execution. The update rule: $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$ only replaces $D[i][j]$ with a smaller (or equal) value, so entries can only decrease or remain the same.
- **(c) True:** The algorithm correctly handles graphs with negative edge weights, as long as there are no negative weight cycles.
- **(d) False:** The recurrence is not based on a greedy choice. It follows a dynamic programming approach, considering whether including an intermediate vertex k improves the shortest path.

6. Let $G = (V, E)$ be a directed flow network with a single source s and a single sink t , and positive integer capacities on all edges. Let f^* be a maximum s - t flow in G .

A cut (S, T) is a partition of V such that $s \in S$ and $t \in T$.

Suppose we select exactly one edge $e \in E$ and increase its capacity by 1 unit.

Under which of the following conditions is the new maximum flow of the network **strictly guaranteed** to be greater than $|f^*|$? _____

- (A) Edge e is saturated in the current maximum flow f^* .
- (B) Edge e belongs to the set of edges from S to T for at least one minimum s - t cut (S, T) .
- (C) Edge e belongs to the set of edges from S to T for every minimum s - t cut (S, T) .
- (D) Edge e belongs to the set of edges from S to T for at least one minimum s - t cut (S, T) and is saturated in f^* .

Solution: C

Explanation: By the **Max-Flow Min-Cut Theorem**, the value of the maximum flow equals the capacity of a minimum s - t cut:

$$|f^*| = \min_{(S,T)} \text{cap}(S, T).$$

If we increase the capacity of an edge e by 1, the maximum flow will increase if and only if the capacity of *every* minimum cut increases.

Analysis of options:

- (A) Being saturated does not imply that the edge lies in any minimum cut. Hence, this does not guarantee an increase in max flow.
- (B) If e belongs to at least one minimum cut, increasing its capacity increases the capacity of that cut. However, there may exist another minimum cut not containing e , whose capacity remains unchanged. Thus, the max flow may not increase.
- (C) If e belongs to *every* minimum cut, then increasing its capacity increases the capacity of all minimum cuts. Therefore, the new minimum cut capacity increases by 1, and hence the maximum flow strictly increases.
- (D) Even if e is saturated and belongs to some minimum cut, there may still exist another minimum cut not containing e . Hence, this does not guarantee an increase in max flow.

Correct answer: (C)

7. Given a directed graph $G = (V, E)$, determine the path from S to D and minimum time T such that there exists a path from source S to destination D .

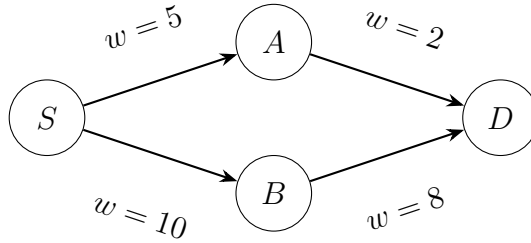
Each edge $e_i \in E$ has an associated weight w_{e_i} . However, unlike standard shortest path problems, the time required to traverse an edge depends on the arrival time at that edge.

If a traveler reaches edge e_i at time t_i , then the delay incurred in traversing that edge is given by:

$$\Delta t_i = w_{e_i} + (t_i \bmod w_{e_i}) \quad (1)$$

$$T = \sum \Delta t_i$$

State Visualization



Start at $t = 0$

Solution: Path: $S \rightarrow A \rightarrow D$, **T:** 8

Explanation:

Problem Description

We are given a directed graph $G = (V, E)$ where each edge e_i has a weight w_e . Unlike standard shortest path problems, the traversal time depends on the arrival time.

If a traveler reaches edge e_i at time t_i , the delay incurred is:

$$\Delta t_i = w_e + (t_i \bmod w_e)$$

Total travel time:

$$T = \sum \Delta t_i$$

The journey starts at time:

$$t = 0$$

Possible Paths

From the graph, there are two paths from S to D :

- Path 1: $S \rightarrow A \rightarrow D$
- Path 2: $S \rightarrow B \rightarrow D$

Edge weights:

$$\begin{aligned}w(S, A) &= 5, & w(A, D) &= 2 \\w(S, B) &= 10, & w(B, D) &= 8\end{aligned}$$

Path 1: $S \rightarrow A \rightarrow D$

Step 1: $S \rightarrow A$

$$\begin{aligned}t &= 0, & w &= 5 \\ \Delta t_1 &= 5 + (0 \bmod 5) = 5\end{aligned}$$

New time:

$$t = 5$$

Step 2: $A \rightarrow D$

$$\begin{aligned}t &= 5, & w &= 2 \\ 5 \bmod 2 &= 1 \\ \Delta t_2 &= 2 + 1 = 3\end{aligned}$$

New time:

$$t = 8$$

Total Time

$$T = 5 + 3 = 8$$

Path 2: $S \rightarrow B \rightarrow D$

Step 1: $S \rightarrow B$

$$\begin{aligned}t &= 0, & w &= 10 \\ \Delta t_1 &= 10 + (0 \bmod 10) = 10\end{aligned}$$

New time:

$$t = 10$$

Step 2: $B \rightarrow D$

$$t = 10, \quad w = 8$$

$$10 \bmod 8 = 2$$

$$\Delta t_2 = 8 + 2 = 10$$

New time:

$$t = 20$$

Total Time

$$T = 10 + 10 = 20$$

Final Answer

Optimal Path: $S \rightarrow A \rightarrow D$

Minimum Time: $T = 8$

Key Insights

- Edge traversal time depends on arrival time.
- This is a **time-dependent shortest path problem**.
- Standard shortest path algorithms must be modified to account for dynamic edge costs.
- The optimal solution requires simulating time along each path.

8. Consider a directed graph $G = (V, E)$ with n vertices and m edges ($|V| = n, |E| = m$), which may contain negative edge weights but is guaranteed to have **no negative cycles**. You are comparing two Dynamic Programming (DP) approaches to solve the All-Pairs Shortest Path (APSP) problem:
- (a) **The “Edge-Budget” Approach:** $A[i, j, k]$ represents the shortest path from i to j using at most k edges.
 - (b) **The “Prefix/Intermediate” Approach (Floyd-Warshall):** $A[i, j, k]$ represents the shortest path from i to j using only internal vertices from the set $V^{(k)} = \{1, 2, \dots, k\}$.

In a **dense graph** where the number of edges $m \approx n^2$, which of the following statements is **TRUE** regarding their computational efficiency? _____

- A) Both approaches achieve a total runtime of $O(n^3)$ because they both solve n^2 subproblems for each of the n values of k .
- B) The “Edge-Budget” approach is asymptotically slower, requiring $O(n^4)$ time, because calculating each state $A[i, j, k]$ requires iterating over the in-neighbors of vertex j for every source i .
- C) The “Prefix” approach is asymptotically slower because the strict dependency on a specific vertex ordering $\{1, 2, \dots, k\}$ adds a logarithmic overhead to each subproblem update.
- D) Both approaches are equivalent in time complexity, but the “Edge-Budget” approach is preferred for dense graphs because it uses $O(n^2)$ space while the “Prefix” approach requires $O(n^3)$ space.

Solution: B

Explanation:

Problem Overview

We are given a directed graph $G = (V, E)$ with:

- $|V| = n$ vertices
- $|E| = m$ edges

The graph may contain negative edge weights but has no negative cycles.

We compare two Dynamic Programming (DP) approaches for solving the **All-Pairs Shortest Path (APSP)** problem:

- (a) **Edge-Budget Approach**
- (b) **Prefix (Floyd-Warshall) Approach**

We analyze their time complexity for a **dense graph**, where:

$$m \approx n^2$$

1. Edge-Budget Approach

Definition

Let:

$A[i, j, k]$ = shortest path from i to j using at most k edges

Recurrence

$$A[i, j, k] = \min \left(A[i, j, k-1], \min_{(u,j) \in E} (A[i, u, k-1] + w(u, j)) \right)$$

Complexity Analysis

- Total states: $n^2 \times n = n^3$
- For each state (i, j, k) :
 - We iterate over incoming neighbors of j
 - In a dense graph: degree $\approx n$

Naive Complexity:

$$O(n^3 \cdot n) = O(n^4)$$

2. Prefix (Floyd-Warshall) Approach

Definition

Let:

$A[i, j, k]$ = shortest path from i to j using only intermediate vertices $\{1, 2, \dots, k\}$

Recurrence

$$A[i, j, k] = \min (A[i, j, k-1], A[i, k, k-1] + A[k, j, k-1])$$

Complexity Analysis

- Three nested loops:
 - $k = 1 \dots n$
 - $i = 1 \dots n$
 - $j = 1 \dots n$

Total Complexity: $O(n^3)$

Comparison of Approaches

Approach	State Meaning	Time Complexity
Edge-Budget	Paths with $\leq k$ edges	$O(n^4)$
Prefix (Floyd-Warshall)	Intermediate vertices $\{1, \dots, k\}$	$O(n^3)$

This explains why **Option B** is correct. The Edge-Budget approach is asymptotically slower, with a time complexity of $O(n^4)$, because for each dynamic programming state (i, j, k) , we must iterate over all incoming neighbors of vertex j .

Since there are $O(n^3)$ such states (for all choices of i , j , and k), and each state may require up to $O(n)$ work in a dense graph (due to iterating over neighbors), the total time complexity becomes:

$$O(n^3) \times O(n) = O(n^4).$$

Thus, the repeated iteration over neighbors for every source-destination pair at every edge-count step leads to the higher asymptotic complexity.

9. In a flow network, let p be an augmenting path in the residual graph G_f . If we augment the flow along p by its bottleneck capacity

$$b = \min_{e \in p} c_f(e),$$

which of the following is **always true**? _____

- (A) Exactly one edge becomes saturated
- (B) At least one edge becomes saturated
- (C) All edges become saturated
- (D) No edge becomes saturated

Solution: B

Explanation:

By definition, the bottleneck capacity is

$$b = \min_{e \in p} c_f(e),$$

i.e., the minimum residual capacity among all edges on the path p .

When we augment the flow by b , for each edge e in p , its residual capacity decreases by b :

$$c_f(e) \leftarrow c_f(e) - b.$$

Since b is the minimum residual capacity along the path, there exists at least one edge e^* such that

$$c_f(e^*) = b.$$

After augmentation:

$$c_f(e^*) = 0,$$

so this edge becomes saturated.

If multiple edges have residual capacity equal to b , then all of them become saturated.

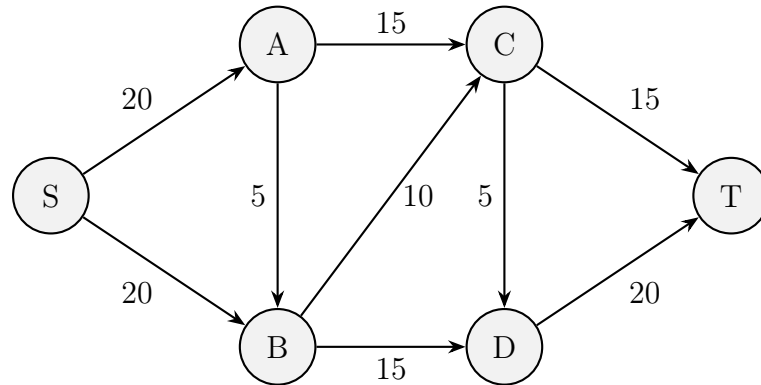
Hence, **at least one edge becomes saturated**, but not necessarily exactly one.

Correct answer: (B)

10. Consider the directed flow network $G = (V, E)$ shown below. The source is node S and the sink is node T . Each edge (u, v) is labeled with its capacity $c(u, v)$.

Using the Ford-Fulkerson method, suppose the following augmenting path is selected in the first iteration:

$$S \rightarrow A \rightarrow C \rightarrow T$$



Assume that the initial flow in the network is zero.

Note: The flow sent along an augmenting path is equal to the minimum capacity among all edges on that path.

- Determine the maximum flow that can be sent through this path. _____
- After sending this flow, compute the updated residual capacities of edge (A, C)

- Specify the capacity of the reverse edge (T, C) created in the residual graph

Solution: 15, 0, 15

Explanation:

Problem Description

We are given a directed flow network $G = (V, E)$ with source S and sink T . Each edge (u, v) has capacity $c(u, v)$. The initial flow is zero.

The augmenting path selected in the first iteration is:

$$S \rightarrow A \rightarrow C \rightarrow T$$

The capacities along this path are:

$$c(S, A) = 20, \quad c(A, C) = 15, \quad c(C, T) = 15$$

(a) Maximum Flow Through the Path

In the Ford–Fulkerson method, the flow sent along an augmenting path is equal to the minimum capacity (bottleneck) along the path.

$$\begin{aligned}\text{Flow} &= \min\{c(S, A), c(A, C), c(C, T)\} \\ &= \min\{20, 15, 15\} = 15\end{aligned}$$

Maximum flow through the path = 15

(b) Updated Residual Capacity of Edge (A, C)

The residual capacity of a forward edge after sending flow is given by:

$$c_f(u, v) = c(u, v) - f(u, v)$$

For edge (A, C) :

$$c_f(A, C) = 15 - 15 = 0$$

Residual capacity of $(A, C) = 0$

(c) Capacity of Reverse Edge (T, C)

When flow is sent along an edge, a reverse edge is created in the residual graph.

The capacity of the reverse edge is equal to the flow sent:

$$c_f(v, u) = f(u, v)$$

For edge (C, T) :

$$c_f(T, C) = 15$$

Capacity of reverse edge $(T, C) = 15$

Final Answers

- (a) 15
- (b) 0
- (c) 15

Key Concepts

- Flow along a path is determined by the **minimum capacity** (bottleneck).
- Residual capacity of forward edge: $c(u, v) - f(u, v)$.
- Residual capacity of reverse edge: $f(u, v)$.
- Reverse edges allow the algorithm to adjust (reduce) flow in future iterations.